ESO

External Secrets Operator

A practical guide — Beginner to Expert

Covers ESO v1 (latest) · CNCF Sandbox Project

Table of Contents

1. Introduction — What is External Secrets Operator?

External Secrets Operator (ESO) is a Kubernetes controller that reads secrets from external secret-management systems — AWS Secrets Manager, Azure Key Vault, HashiCorp Vault, Google Secret Manager, 1Password, GitHub, and dozens more — and synchronizes them into native Kubernetes Secret resources. Your applications keep consuming standard Kubernetes Secrets via environment variables or volume mounts; they have no idea that the source of truth lives outside the cluster.

ESO started life in 2019 as a community project, joined the CNCF Sandbox in 2022, and is on the path to Incubation. It is maintained by an active community of contributors from companies that run it in production at significant scale — Container Solutions, Cyral, Form3, and others.

The one-line definition

> 🎯 **ESO in one sentence**
>
> ESO is a Kubernetes operator that turns secrets from your external vault into ordinary Kubernetes Secrets, automatically and continuously, so applications stay decoupled from the vault.

What it is NOT

A few clarifications upfront, because misconceptions are common:

- **Not a secret store.** ESO does not store secrets. The vault (AWS, Azure, Vault, etc.) does. ESO is a pipe.
- **Not an encryption tool.** Secrets land in etcd encrypted at rest only if you configure encryption-at-rest on the cluster yourself (KMS provider, Sealed Secrets are an alternative for that).
- **Not a replacement for Vault.** ESO consumes from Vault; it does not replace it. Keep Vault for rotation, dynamic secrets, and audit; use ESO to deliver to pods.
- **Not magic for secret rotation.** ESO pulls on a refresh interval. If the upstream secret rotates, ESO updates the Kubernetes Secret — but your pods still need to re-read it, which usually requires a restart unless they hot-reload.

2. Why ESO? — The Problem with Kubernetes Secrets

Kubernetes Secrets are convenient and dangerous in equal measure. They look like ConfigMaps with base64 encoding — and base64 is not encryption, it is decoration. Without operational discipline, secrets end up committed to Git, copy-pasted into Slack, screenshotted in incident channels, and replicated across thirty namespaces. ESO exists to break that cycle.

The five failure modes ESO solves

Problem 1 — Secrets in Git

The most common security incident in any growing engineering org: a developer commits config.yaml with the database password in it, the repo is scanned by a bot fifteen minutes later, and Friday afternoon becomes a credential-rotation marathon. ESO eliminates the temptation: there is nothing to commit. The Git manifest references an external key by name, never the value.

Problem 2 — Rotation without redeploy

Your DB password rotates every 30 days. Without ESO, you redeploy every workload that uses it — a coordination headache across teams. With ESO, you rotate in the vault, ESO picks up the change on its next refresh, and the Kubernetes Secret is updated automatically. Pods can pick it up via reloader, sidecars, or restart.

Problem 3 — Multi-cluster sprawl

You run three clusters: dev, staging, prod. Each needs the same database connection string. Without ESO you either (a) copy-paste secrets three times and forget which is current, or (b) build a custom sync tool. With ESO you create one SecretStore per cluster pointing at the same vault; the secrets stay in sync everywhere automatically.

Problem 4 — Separation of duties

Compliance and security teams want central control of credentials — who can read what, who can rotate what, full audit. Developers want to use credentials without filing tickets. ESO splits these concerns cleanly: security owns the vault and the SecretStore; developers write ExternalSecret manifests describing what they need. Two different RBAC scopes, one workflow.

Problem 5 — Inconsistent ergonomics across vaults

Every secret vault has its own SDK, auth model, and API quirks. Hardcoding HashiCorp Vault client code in your Go service and AWS SDK code in your Python service and Azure SDK code in your Node.js service is repetitive and error-prone. ESO provides one consistent interface — the Kubernetes Secret — regardless of which vault is on the other end.

> 💡 **TIP**
> The biggest win from adopting ESO is usually not technical; it is organizational. Once secrets live in a vault outside Git, the conversation about who owns rotation, audit, and break-glass becomes much easier to have.

Use cases at a glance

Scenario	What You Configure	Vault
App reads DB password from env var	ExternalSecret → Secret → env	AWS Secrets Manager / Vault

Scenario	What You Configure	Vault
TLS cert lifecycle from cert-manager	Push cert to vault for backup	Azure Key Vault
Multi-cluster GitOps with shared creds	ClusterSecretStore per cluster	Vault / GCP SM
CI/CD shared secrets to GitHub Actions	PushSecret to GitHub	GitHub Actions Secrets
Bootstrap kubeconfig credentials	ExternalSecret with templating	Vault KV2
Per-team isolated secret access	Namespaced SecretStore + RBAC	Any
Pull-secrets for private registries	ECR generator / dockerconfig template	AWS ECR / AKS / GCR
Random password generation	Password generator + PushSecret	AWS / Azure / Vault

3. Architecture & Core Concepts

ESO is a small set of Kubernetes controllers that watch CRDs and reconcile them into Kubernetes Secrets. The mental model is the same as cert-manager: declarative resources describe desired state, controllers do the work, and the result is a regular Kubernetes resource your apps already know how to consume.

The reconciliation loop

1. You create an ExternalSecret that says "I want a Kubernetes Secret called db-creds, and the value should come from the vault path prod/db".
2. The ESO controller wakes up (event-driven on resource changes, plus a periodic refreshInterval) and reads the ExternalSecret.
3. It looks up the referenced SecretStore to learn how to authenticate to the vault.
4. It calls the vault API to fetch the value.
5. It applies optional templating, decoding, key rewriting.
6. It creates or updates the Kubernetes Secret with the result. Your pods pick it up via env or volume mount.
7. It re-runs on every refreshInterval. If the upstream value changes, the Kubernetes Secret is updated; if not, nothing happens.

Runtime components

Core controller (external-secrets)

The main Deployment. Watches all ESO CRDs, calls vault APIs, creates Kubernetes Secrets. The bulk of the operator lives here.

Webhook (external-secrets-webhook)

A validating admission webhook. Catches malformed ExternalSecret and SecretStore manifests before they enter etcd — for example, a SecretStore referencing a secret in another namespace, which would violate the namespace isolation contract.

Cert controller (external-secrets-cert-controller)

Manages the TLS certificate used by the webhook. Self-rotating, no external CA needed. You can replace it with cert-manager if you prefer.

Personas — who does what

Persona	Owns	Typical Resources They Touch
Cluster operator / platform team	Installation, ClusterSecretStores, RBAC	ClusterSecretStore, the operator itself
Security / vault admin	Vault policies, IAM roles, audit	Vault paths, IAM policies, Service Principals
Application developer	ExternalSecrets in their own namespaces	ExternalSecret, SecretStore (optional)

The cleanest split is: security owns "how" (SecretStore + vault credentials); developers own "what" (ExternalSecret pointing at the secret name they need).

4. The Resource Model — CRDs at a Glance

ESO ships six core CRDs. Two are absolutely essential (SecretStore and ExternalSecret); the rest cover cluster-wide scope, the reverse flow, and generation. Once you know these six, you know 90% of ESO.

CRD	Scope	Purpose
SecretStore	Namespaced	How to authenticate to a vault. Owned by app teams or platform.
ClusterSecretStore	Cluster-wide	Same as SecretStore, but referenceable from every namespace.
ExternalSecret	Namespaced	What secrets to pull and into which K8s Secret. The bread and butter.
ClusterExternalSecret	Cluster-wide	Apply the same ExternalSecret across many namespaces (label-selected).
PushSecret	Namespaced	Reverse flow: push from K8s Secret to the vault.
ClusterPushSecret	Cluster-wide	Push across many namespaces.

A minimum working example

Three resources: a Secret holding vault credentials, a SecretStore using those credentials, and an ExternalSecret declaring what to pull. This is the canonical "hello world".

```
# 1. Credentials for the vault, stored as a normal K8s Secret
apiVersion: v1
kind: Secret
metadata:
  name: aws-creds
  namespace: my-app
stringData:
  access-key: AKIA...
  secret-access-key: ...

---
# 2. SecretStore — how to talk to AWS Secrets Manager
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: aws-store
  namespace: my-app
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-central-1
```

```
      auth:
        secretRef:
          accessKeyIDSecretRef:
            name: aws-creds
            key: access-key
          secretAccessKeySecretRef:
            name: aws-creds
            key: secret-access-key

---
# 3. ExternalSecret — what to pull and where to put it
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: my-app-secrets
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-store
    kind: SecretStore
  target:
    name: my-app-secrets        # The K8s Secret to create
    creationPolicy: Owner
  data:
  - secretKey: DB_PASSWORD       # Key inside the K8s Secret
    remoteRef:
      key: prod/my-app/db        # Path in AWS Secrets Manager
      property: password         # Field within that secret (if JSON)
```

Apply that, wait one reconciliation cycle, and kubectl get secret my-app-secrets shows a normal Kubernetes Secret with DB_PASSWORD set to whatever lives in AWS. Your Deployment just envFrom: [secretRef: my-app-secrets] like any other secret.

5. Installation & Deployment

ESO ships as a Helm chart. Installation is one command; there are no operator marketplace integrations to worry about, no managed-add-on flavor to choose from.

Helm install — the standard way

```
# Add the chart repo
helm repo add external-secrets https://charts.external-secrets.io
helm repo update

# Install into its own namespace
helm install external-secrets external-secrets/external-secrets \
  -n external-secrets --create-namespace \
  --set installCRDs=true

# Verify
kubectl get pods -n external-secrets
# Expected:
#   external-secrets-xxx              Running
#   external-secrets-cert-controller-xxx   Running
#   external-secrets-webhook-xxx           Running
```

Production-ready values

The default chart values are fine for a quick demo but should be tightened before production. The skeleton below shows the levers worth pulling.

```
# values.yaml
replicaCount: 2                          # HA
priorityClassName: system-cluster-critical
resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits:   { cpu: 500m, memory: 512Mi }

# Pod security
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities: { drop: ["ALL"] }

# Metrics
serviceMonitor:
  enabled: true                          # If you run kube-prometheus-stack
  interval: 30s

# Disable unused features (security best practice — see §18)
```

```
# processClusterExternalSecret: false
# processClusterStore: false
# processPushSecret: false

# Webhook
webhook:
  replicaCount: 2
  extraArgs:
    tls-ciphers: "TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256"

# If using cert-manager instead of the built-in cert-controller:
# certController:
#   create: false
# webhook:
#   certManager:
#     enabled: true
```

Air-gapped / private registry

ESO publishes signed images to ghcr.io and a mirror to a few cloud registries. For air-gapped clusters, mirror the three images (controller, webhook, cert-controller) to your registry and set image.repository accordingly. Cosign signatures (see §18) verify image integrity offline.

Uninstall

```
# Uninstall the chart
helm uninstall external-secrets -n external-secrets

# CRDs are intentionally NOT removed by helm uninstall —
# this prevents accidental loss of ExternalSecret resources.
# To fully wipe:
kubectl delete crd \
  externalsecrets.external-secrets.io \
  secretstores.external-secrets.io \
  clustersecretstores.external-secrets.io \
  clusterexternalsecrets.external-secrets.io \
  pushsecrets.external-secrets.io \
  clusterpushsecrets.external-secrets.io
```

⚠ WARNING

Deleting the CRDs deletes all ExternalSecret resources. The Kubernetes Secrets they created are kept by default (creationPolicy: Owner means the Secret is GC'd only when the ExternalSecret is deleted — but the CRD removal cascades deletion of all instances).

Back up your ExternalSecret manifests in Git before deleting CRDs. Better yet, do not delete the CRDs.

6. SecretStore — How to Access

The SecretStore is the contract between ESO and one specific vault. It says "this is the URL, this is how to authenticate, this is the scope". Each provider (AWS, Azure, Vault, ...) has its own provider sub-block, but the surrounding shape is the same.

Skeleton

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: my-store
  namespace: my-app          # Namespaced — only my-app can use it
spec:
  controller: ""             # Optional. Only handle stores matching a class — see
§17
  retrySettings:             # Optional. Backoff on vault errors
    maxRetries: 5
    retryInterval: "10s"
  refreshInterval: 0         # Optional. 0 = only refresh on ExternalSecret change
  provider:
    <providerName>:          # aws, azurekv, vault, github, gcpsm, kubernetes, ...
      # provider-specific config goes here
```

Key fields

Field	Default	Purpose
provider.<name>	(required)	Which vault. Exactly one provider per store.
controller	""	Tag for multi-instance ESO (see Controller Classes in §17).
retrySettings.maxRetries	0	How many times to retry transient vault errors.
retrySettings.retryInterval	5s	Backoff between retries.
refreshInterval	0	How often to revalidate auth (not data — that is per-ExternalSecret).
conditions (ClusterSecretStore only)	(none)	Restrict which namespaces can use this store.

Authentication model

Every provider supports at least one of these auth flavors. Choose based on where ESO runs and your existing identity stack:

- **Static credentials in a Secret:** Easy to set up, hard to rotate. Use only for non-production or where nothing else fits.
- **Cloud workload identity (IRSA, AKS Workload Identity, GCP WI):** No long-lived secrets. The strongly recommended default in cloud-managed Kubernetes.

- **Kubernetes ServiceAccount JWT:** For HashiCorp Vault Kubernetes auth, Conjur, etc. ESO mints short-lived SA tokens via the TokenRequest API.
- **AppRole / Token / OIDC / Cert:** Vault-specific options. AppRole is the most common for K8s-to-Vault.

> 💡 **TIP**
> A common anti-pattern is "one giant ClusterSecretStore that everyone uses". It makes blast-radius huge: a single misconfigured ExternalSecret can read anything. Prefer many small namespaced SecretStores, each pointing at the smallest vault scope each team needs.

7. ExternalSecret — What to Fetch

The ExternalSecret is the developer-facing CRD. It says: "Take these named keys from the vault, do these transformations, and produce this Kubernetes Secret." There are three flavors of data declaration: data, dataFrom.extract, and dataFrom.find. Pick based on whether you know the names ahead of time.

Full spec — annotated

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: my-app-secrets
  namespace: my-app
spec:
  refreshInterval: "1h"                  # How often to re-pull from the vault
  secretStoreRef:
    name: aws-store                      # Must exist in same namespace (or kind:
ClusterSecretStore)
    kind: SecretStore

  target:
    name: my-app-secrets                 # K8s Secret to create
    creationPolicy: Owner                # Owner | Orphan | Merge | None
    deletionPolicy: Retain               # Retain | Delete | Merge
    template:                            # Optional — see §15 (Templating)
      type: Opaque                       # Or kubernetes.io/tls,
kubernetes.io/dockerconfigjson, ...
      engineVersion: v2
      metadata:
        labels: { managed-by: eso }
      data:
        config.yaml: |
          database:
            host: {{ .DB_HOST }}
            password: {{ .DB_PASSWORD }}

  data:
  - secretKey: DB_HOST                   # Field name in K8s Secret
    remoteRef:
      key: prod/my-app/db                # Path in the vault
      property: host                     # Nested field (JSON or KV2)
      version: AWSCURRENT                # Optional version (AWS only)
      decodingStrategy: None             # Auto | Base64 | Base64URL | None
  - secretKey: DB_PASSWORD
    remoteRef:
      key: prod/my-app/db
      property: password

  dataFrom:
```

```
  - extract:                          # Pull every field of one vault entry
      key: prod/my-app/api-creds
      conversionStrategy: Default     # Default | Unicode
      decodingStrategy: None
  - find:                             # Pull every secret matching a pattern
      name:
        regexp: "^prod/my-app/"
      path: "/"
      tags:
        team: backend
```

creationPolicy — who owns the resulting Secret

Policy	Behavior
Owner (default)	ESO creates and owns the Secret. Deleting the ExternalSecret deletes the Secret.
Orphan	ESO creates but does not own. Useful when other tools need the Secret to outlive ESO.
Merge	Do NOT create — only update keys in an existing Secret. Used when the Secret is bootstrapped elsewhere.
None	Do not create or update. Combined with template-only flows (rare).

deletionPolicy — what happens when the vault entry disappears

Policy	Behavior
Retain (default)	Keep the last-known value in the K8s Secret. Safest — avoids accidental outages.
Delete	Delete the K8s Secret when the vault entry is gone. Use when missing secret = no service.
Merge	Just remove the deleted keys, keep others.

data vs dataFrom

Use data when you know the exact keys you want and how to name them. Use dataFrom when you want bulk — every field of a JSON blob, or every secret matching a pattern. dataFrom is convenient but dangerous: a vault admin renaming a field can quietly change a Secret. For production, explicit data declarations are usually safer.

refreshInterval — the polling knob

How often ESO re-reads the vault for this ExternalSecret. Default is 1h. Shorter (5m, 15m) means faster rotation pickup but more vault API calls. Setting it to "0" disables polling entirely — ESO only refreshes when the ExternalSecret itself changes. Useful when you have an external trigger (rotation webhook calling kubectl annotate).

> ⚠ **WARNING**
> Cloud-provider APIs cost real money on high volume. A 1m refreshInterval across 200 ExternalSecrets is 12,000 API calls per hour. AWS Secrets Manager charges per 10,000 API calls. Do the math before going aggressive.

8. ClusterSecretStore & ClusterExternalSecret

Namespaced SecretStore + ExternalSecret is the right default for most teams. But sometimes you genuinely need cross-namespace behavior — a single central credentials story or a fleet-wide config secret. That is what the cluster-scoped variants are for.

ClusterSecretStore — one store, many namespaces

Identical to SecretStore except (a) it has no namespace and (b) it can be referenced from any namespace by setting secretStoreRef.kind: ClusterSecretStore. Use sparingly — the blast radius is the whole cluster.

```
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: vault-shared
spec:
  conditions:                          # CRITICAL: restrict which namespaces can
use this
  - namespaceSelector:
      matchLabels:
        eso-allowed: "true"
  # ...alternatively:
  # - namespaces:                      # Explicit list of namespaces
  #     - team-a
  #     - team-b
  provider:
    vault:
      server: "https://vault.example.com"
      path: "kv"
      version: "v2"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "eso-reader"
          serviceAccountRef:
            name: external-secrets
            namespace: external-secrets
```

⚠ WARNING

A ClusterSecretStore with NO conditions block is usable from any namespace. Anyone with permission to create an ExternalSecret in any namespace can read anything the store can read.

Always set conditions. Treat ClusterSecretStore as a privileged resource and gate it behind namespace labels controlled by the platform team.

ClusterExternalSecret — fan-out one definition to many namespaces

Have 40 namespaces that all need the same set of secrets (e.g., a shared "log-shipper-creds" Secret for every team)? Without ClusterExternalSecret you write 40 manifests. With it, you write one and select target namespaces via label selector.

```yaml
apiVersion: external-secrets.io/v1
kind: ClusterExternalSecret
metadata:
  name: log-shipper-creds
spec:
  externalSecretName: log-shipper-creds    # Name of the ExternalSecret to create
in each ns
  namespaceSelector:
    matchLabels:
      tier: production                     # Only deploy into prod namespaces
  refreshTime: 1h
  externalSecretSpec:
    refreshInterval: 1h
    secretStoreRef:
      name: vault-shared
      kind: ClusterSecretStore
    target:
      name: log-shipper-creds
    data:
    - secretKey: API_KEY
      remoteRef:
        key: shared/log-shipper
        property: api-key
```

ESO watches namespaces matching the selector. Label a namespace tier=production after the fact and ESO automatically creates the ExternalSecret there. Remove the label and the ExternalSecret (and its Secret) goes away.

9. PushSecret — The Reverse Flow

ESO is bidirectional. ExternalSecret pulls from vault → Kubernetes; PushSecret pushes Kubernetes → vault. Why would you want that? Three big reasons.

Why push?

8. Bootstrapping a vault from existing K8s Secrets when adopting ESO — migrate the data over without copy-pasting.
9. Centralizing secrets generated inside the cluster (cert-manager certs, randomly generated passwords, tokens minted by an app) into the vault for backup and cross-cluster reuse.
10. Syncing to write-only endpoints like GitHub Actions Secrets, which can be written via API but never read back.

Example: cert-manager → Azure Key Vault

Scenario: cert-manager issues a TLS cert via Let's Encrypt and stores it in a K8s Secret. You want a copy in Azure Key Vault so other services (an Azure Front Door, a webhook receiver) can use the same cert without going through Kubernetes.

```
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: push-ingress-cert
  namespace: ingress
spec:
  refreshInterval: 1h
  deletionPolicy: Delete                # Delete from AKV if K8s Secret goes
away
  secretStoreRefs:
  - name: azure-store
    kind: SecretStore
  selector:
    secret:
      name: ingress-tls                 # Source K8s Secret (managed by cert-
manager)
  data:
  - match:
      secretKey: tls.crt
      remoteRef:
        remoteKey: cert/ingress-tls     # cert/ prefix = certificate in AKV
    metadata:
      apiVersion: kubernetes.external-secrets.io/v1alpha1
      kind: PushSecretMetadata
      spec:
        tags:
          source: cert-manager
          environment: production
```

Example: write-only sync to GitHub Actions Secrets

Scenario: Your CI workflows need a Docker Hub token to push images. Storing it as a GitHub repository secret is the natural place. You manage all org secrets in Vault for centralized rotation. ESO can keep them in sync — Vault is the source of truth, GitHub is a destination.

```
# First, an ExternalSecret pulls the token from Vault into K8s
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: docker-token
  namespace: ci
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: vault-store
    kind: SecretStore
  target:
    name: docker-token
  data:
  - secretKey: token
    remoteRef:
      key: shared/ci/dockerhub
      property: token

---
# Then, a PushSecret writes it to GitHub Actions
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: push-docker-token
  namespace: ci
spec:
  refreshInterval: 1h
  deletionPolicy: Delete
  secretStoreRefs:
  - name: github-store
    kind: SecretStore
  selector:
    secret:
      name: docker-token
  data:
  - match:
      secretKey: token
      remoteRef:
        remoteKey: DOCKERHUB_TOKEN        # Name as it appears in GitHub Secrets
UI
```

📝 **NOTE**

> The GitHub provider is write-only by design — the GitHub REST API does not expose secret values for reading. ESO can create and update them; nothing can read them back, including ESO itself.

10. AWS Secrets Manager — End-to-End

AWS Secrets Manager is the most common provider in cloud-managed Kubernetes deployments. The recommended setup uses IRSA (IAM Roles for Service Accounts) — no long-lived AWS keys, just a federated identity. We walk through the full path: IAM policy, role, ServiceAccount, SecretStore, ExternalSecret.

Step 1 — IAM policy

Grant the minimum permissions ESO needs. Scope by ARN pattern; do not grant Resource: "*" in production.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue",
        "secretsmanager:DescribeSecret",
        "secretsmanager:ListSecretVersionIds"
      ],
      "Resource": [
        "arn:aws:secretsmanager:eu-central-1:123456789012:secret:prod/my-app/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:ListSecrets",
        "secretsmanager:BatchGetSecretValue"
      ],
      "Resource": "*"
    }
  ]
}
```

TIP

Use secretsmanager:BatchGetSecretValue (released Nov 2023) wherever possible — one API call returns up to 20 secrets. Define a "path" in your SecretStore to enable batching, or use the Tags filter on dataFrom.find. Cuts AWS bill on busy clusters dramatically.

Step 2 — IAM role with trust policy

```
# Trust policy attached to the role — allows the K8s ServiceAccount to assume it
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
```

```
    "Principal": {
      "Federated": "arn:aws:iam::123456789012:oidc-provider/oidc.eks.eu-central-
1.amazonaws.com/id/EXAMPLE"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "oidc.eks.eu-central-1.amazonaws.com/id/EXAMPLE:sub":
          "system:serviceaccount:my-app:external-secrets-sa",
        "oidc.eks.eu-central-1.amazonaws.com/id/EXAMPLE:aud": "sts.amazonaws.com"
      }
    }
  }]
}
```

Step 3 — ServiceAccount, SecretStore, ExternalSecret

```
# ServiceAccount in the workload namespace, annotated with the role ARN
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets-sa
  namespace: my-app
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/eso-my-app

---
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: aws-store
  namespace: my-app
spec:
  provider:
    aws:
      service: SecretsManager
      region: eu-central-1
      auth:
        jwt:
          serviceAccountRef:
            name: external-secrets-sa     # Uses IRSA — no static creds!

---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-creds
  namespace: my-app
```

```
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-store
    kind: SecretStore
  target:
    name: db-creds
  data:
  - secretKey: username
    remoteRef:
      key: prod/my-app/db
      property: username                # AWS SM stores JSON; pull one field
  - secretKey: password
    remoteRef:
      key: prod/my-app/db
      property: password
```

AWS Secrets Manager features worth knowing

- **JSON secrets:** AWS SM stores secret values as opaque blobs but treats JSON specially. Use property to extract one field. Supports gjson syntax for nested paths: friendslist.0.first.
- **Versions:** Every update creates a new version. By default ESO reads AWSCURRENT. Use version: "AWSPREVIOUS" to read the previous one — handy during rotation cutover. Or version: "uuid/<uuid>" to pin to an immutable version.
- **AWS Parameter Store:** Cheaper alternative for non-rotating config. Use service: ParameterStore in the same provider block.
- **PushSecret support:** Yes. Tags, KMS keys, descriptions, resource policies all configurable via metadata.

11. Azure Key Vault — End-to-End

Azure Key Vault (AKV) handles three distinct object types — secrets, keys, and certificates — and ESO supports all three. On AKS the recommended authentication is Workload Identity (federated OIDC); for non-Azure clusters, Service Principal with a client secret or certificate is the fallback.

Step 1 — Enable AKS Workload Identity

```
# Create cluster with Workload Identity (or update existing)
az aks update -g <rg> -n <cluster> --enable-oidc-issuer --enable-workload-identity

# Capture the OIDC issuer URL
AKS_OIDC_ISSUER=$(az aks show -g <rg> -n <cluster> --query
oidcIssuerProfile.issuerUrl -o tsv)

# Create a user-assigned managed identity
az identity create -g <rg> -n eso-identity
CLIENT_ID=$(az identity show -g <rg> -n eso-identity --query clientId -o tsv)
PRINCIPAL_ID=$(az identity show -g <rg> -n eso-identity --query principalId -o
tsv)

# Grant it Key Vault Secrets User on the vault
az role assignment create \
  --assignee-object-id "$PRINCIPAL_ID" \
  --role "Key Vault Secrets User" \
  --scope "/subscriptions/<sub>/resourceGroups/<rg>/providers/Microsoft.KeyVault/
vaults/<vault-name>"

# Create the federated credential
az identity federated-credential create -g <rg> -n eso-fc --identity-name eso-
identity \
  --issuer "$AKS_OIDC_ISSUER" \
  --subject "system:serviceaccount:my-app:external-secrets-sa" \
  --audiences api://AzureADTokenExchange
```

Step 2 — Kubernetes resources

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets-sa
  namespace: my-app
  annotations:
    azure.workload.identity/client-id: "<CLIENT_ID-from-above>"
    azure.workload.identity/tenant-id: "<TENANT_ID>"

---
apiVersion: external-secrets.io/v1
```

```
kind: SecretStore
metadata:
  name: azure-store
  namespace: my-app
spec:
  provider:
    azurekv:
      authType: WorkloadIdentity
      vaultUrl: "https://my-prod-vault.vault.azure.net"
      serviceAccountRef:
        name: external-secrets-sa
```

Step 3 — Pull secrets, keys, certs

AKV exposes three object types. Use the prefix in remoteRef.key to pick which one. Default is secret if no prefix is given.

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: app-secrets
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: azure-store
    kind: SecretStore
  target:
    name: app-secrets
    template:
      type: kubernetes.io/tls              # Build a real TLS secret
      engineVersion: v2
      data:
        tls.crt: '{{ .pfx | b64dec | pkcs12cert }}'
        tls.key: '{{ .pfx | b64dec | pkcs12key }}'
  data:
  # 1. A plain secret (no prefix = secret)
  - secretKey: db-password
    remoteRef:
      key: db-password
  # 2. A certificate — fetched as a PKCS#12 blob via secret/ prefix
  #    (AKV stores certs as both a cert object and a hidden secret;
  #    secret/ returns the PFX which we then template above)
  - secretKey: pfx
    remoteRef:
      key: secret/tls-client-cert
  # 3. A public key — key/ prefix returns it as a JWK
  - secretKey: jwt-pubkey
```

```
    remoteRef:
      key: key/jwt-signing-key
```

Prefix in remoteRef.key	AKV Object	What You Get
(none) or secret/	Secret	Raw string value
key/	Key	Public key as a JWK JSON blob (private key not exported)
cert/	Certificate	X.509 cert bytes — use template fns to convert encoding

TIP

AKV certificates have a quirk: a certificate object internally is both a Certificate AND a Secret (the cert object exposes metadata; the linked secret holds the actual PFX bytes). To fetch the full chain + private key, use secret/<name> rather than cert/<name>.

12. HashiCorp Vault — End-to-End

HashiCorp Vault is the heavyweight in this space. ESO supports the KV secrets engine (v1 and v2) and a broad set of auth methods: Token, AppRole, Kubernetes, LDAP, UserPass, JWT/OIDC, AWS IAM, TLS Cert. For Vault running outside Kubernetes, AppRole is the most common pattern; for in-cluster Vault, Kubernetes auth wins.

Kubernetes auth — the natural fit

Vault validates the ServiceAccount JWT via the Kubernetes TokenReview API. No shared secrets in either direction — Vault trusts the cluster's OIDC issuer, the cluster trusts Vault's CA.

```
# Vault side — once, by the Vault admin
vault auth enable kubernetes
vault write auth/kubernetes/config \
  kubernetes_host="https://kubernetes.default.svc"

# Define a role tying a K8s ServiceAccount to a Vault policy
vault write auth/kubernetes/role/eso-my-app \
  bound_service_account_names=external-secrets-sa \
  bound_service_account_namespaces=my-app \
  policies=my-app-read-only \
  ttl=24h \
  audience=vault     # Required for Vault 1.21+

# The policy itself
vault policy write my-app-read-only - <<EOF
path "kv/data/my-app/*" { capabilities = ["read"] }
path "kv/metadata/my-app/*" { capabilities = ["read", "list"] }
EOF

# Kubernetes side
apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets-sa
  namespace: my-app

---
# Cluster admin grants the SA permission to validate its own token via TokenReview
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: eso-my-app-tokenreview
roleRef:
  kind: ClusterRole
  name: system:auth-delegator
  apiGroup: rbac.authorization.k8s.io
subjects:
```

```
- kind: ServiceAccount
  name: external-secrets-sa
  namespace: my-app

---
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: vault-store
  namespace: my-app
spec:
  provider:
    vault:
      server: "https://vault.example.com:8200"
      path: "kv"
      version: "v2"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "eso-my-app"
          serviceAccountRef:
            name: external-secrets-sa
            audiences:
            - vault                        # MUST match the role's audience

---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: my-app-config
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: vault-store
    kind: SecretStore
  target:
    name: my-app-config
  data:
  - secretKey: db_password
    remoteRef:
      key: my-app/database              # vault kv get kv/my-app/database
      property: password                # Nested via gjson — supports my-
app.db.password too
  dataFrom:
  - extract:                            # Pull entire object as flat keys
      key: my-app/api-credentials
```

Vault Enterprise — namespaces and consistency

Vault Enterprise supports multi-tenancy via namespaces. Set provider.vault.namespace on your SecretStore. You can authenticate in one namespace (e.g., a shared auth tenant) and read from a different one (e.g., your team's tenant) by also setting auth.namespace.

AppRole — when Kubernetes auth is not available

When ESO must reach Vault running outside the cluster and you cannot configure Vault's K8s auth method, AppRole is the standard fallback. The roleId is non-sensitive (lives in YAML), the secretId is sensitive (Kubernetes Secret). Rotate secretIds periodically — Vault supports per-secretId TTLs.

```
spec:
  provider:
    vault:
      server: "https://vault.example.com:8200"
      path: "kv"
      version: "v2"
      auth:
        appRole:
          path: "approle"
          roleId: "f0e1d2c3-b4a5-..."          # OK to commit
          secretRef:
            name: "vault-approle-secret-id"     # Sensitive — in a K8s Secret
            key: "secret-id"
```

TIP

Enable the Vault token cache (--enable-vault-token-cache flag) on the ESO operator if you use short-lived auth methods (AppRole, Kubernetes auth). Each ExternalSecret reconciliation otherwise triggers a fresh login. With 200 ExternalSecrets refreshing every 5 minutes, you save thousands of Vault auth calls per hour.

13. GitHub Actions Secrets — End-to-End

The GitHub provider is the odd one out: it is write-only. You cannot pull secrets from GitHub Actions because the API does not expose them. ESO uses it exclusively for PushSecret — taking a value that lives somewhere else (Vault, AWS, a generated password) and putting it where GitHub Actions workflows can consume it.

When to use it

- You manage all secrets centrally in Vault and want CI to consume them without copy-pasting.
- A nightly job in Kubernetes rotates a third-party API key; you need GitHub Actions to receive the new value automatically.
- Cert-manager issues a deploy cert and you want a downstream GitHub workflow to use it without storing it in Git.

Step 1 — Install the ESO GitHub App

The GitHub provider authenticates via a GitHub App, not a personal access token. Install the official ESO app into your organization (or fork the manifest if you need a custom app). Note the App ID and the Installation ID once installed.

Step 2 — Store the App's private key as a Secret

```
# Convert the .pem you downloaded from GitHub into a K8s Secret
kubectl create secret generic github-app-private-key \
  -n my-app \
  --from-file=privateKey.pem=./eso-app.private-key.pem
```

Step 3 — SecretStore

```
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: github-store
  namespace: my-app
spec:
  provider:
    github:
      appID: "123456"                       # From the GitHub App settings
      installationID: "78901234"            # From your org's installations page
      organization: "acme-corp"
      # repository: "my-repo"               # Set for repo-level secrets
      # environment: "production"           # Set for environment-level secrets
      auth:
        privateKey:
          name: github-app-private-key
          key: privateKey.pem
```

Step 4 — PushSecret pushing into a specific repo

```
# Suppose this Secret already exists, populated by another ExternalSecret from
Vault
apiVersion: v1
kind: Secret
metadata:
  name: dockerhub-token
  namespace: my-app
type: Opaque
stringData:
  token: "<populated-by-eso-from-vault>"

---
# Push it to a specific repo's Actions secrets
apiVersion: external-secrets.io/v1
kind: SecretStore                          # Repo-scoped store
metadata:
  name: github-repo-store
  namespace: my-app
spec:
  provider:
    github:
      appID: "123456"
      installationID: "78901234"
      organization: "acme-corp"
      repository: "platform-ci"            # Repo-scoped
      auth:
        privateKey:
          name: github-app-private-key
          key: privateKey.pem

---
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: push-docker-token
  namespace: my-app
spec:
  refreshInterval: 1h
  deletionPolicy: Delete
  secretStoreRefs:
  - name: github-repo-store
    kind: SecretStore
  selector:
    secret:
      name: dockerhub-token
  data:
  - match:
```

```
      secretKey: token
      remoteRef:
        remoteKey: DOCKERHUB_TOKEN          # Name in GitHub UI
```

Org / repo / environment scopes

Scope	SecretStore Fields	Resulting Visibility
Org	organization only	All repos in the org (subject to App permissions)
Repo	organization + repository	Only that repo's workflows
Environment	organization + repository + environment	Only jobs targeting that GitHub Environment

TIP

For deploy-time secrets that need extra protection (production DB passwords, signing keys), push them as Environment secrets rather than Repo secrets. GitHub Environments support required reviewers — even a workflow with full repo access cannot read the secret without the deploy being approved.

14. GCP Secret Manager — Brief Walkthrough

GCP Secret Manager rounds out the big-three cloud providers. The recommended setup uses Workload Identity (the GCP equivalent of IRSA). The pattern is nearly identical to Azure WI — bind a Kubernetes ServiceAccount to a GCP service account via OIDC federation.

```
# Create the GCP service account and grant it Secret Manager access
gcloud iam service-accounts create eso-my-app
gcloud secrets add-iam-policy-binding my-app-secret \
  --member="serviceAccount:eso-my-app@my-proj.iam.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"

# Bind the K8s ServiceAccount to the GCP one via WI
gcloud iam service-accounts add-iam-policy-binding \
  --role roles/iam.workloadIdentityUser \
  --member "serviceAccount:my-proj.svc.id.goog[my-app/external-secrets-sa]" \
  eso-my-app@my-proj.iam.gserviceaccount.com


apiVersion: v1
kind: ServiceAccount
metadata:
  name: external-secrets-sa
  namespace: my-app
  annotations:
    iam.gke.io/gcp-service-account: eso-my-app@my-proj.iam.gserviceaccount.com

---
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: gcp-store
  namespace: my-app
spec:
  provider:
    gcpsm:
      projectID: "my-proj"
      auth:
        workloadIdentity:
          clusterLocation: us-central1
          clusterName: my-gke-cluster
          serviceAccountRef:
            name: external-secrets-sa

---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: my-app-secret
```

```
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: gcp-store
    kind: SecretStore
  target:
    name: my-app-secret
  data:
  - secretKey: api-key
    remoteRef:
      key: my-app-api-key                # GCP secret name
      version: latest                    # Or a specific version number
```

📝 NOTE

GCP Secret Manager versions are immutable numbered objects. version: "latest" follows the latest enabled version; version: "5" pins to a specific version. Pinning is useful when rotating: deploy with version: "5", then bump to "6" once the new version is staged.

15. Advanced Templating

Templating is where ESO transcends "secret syncer" and becomes a secret-composition engine. The template field lets you build dockerconfig JSON, TLS Secrets, multi-line YAML configs — anything Kubernetes can consume — by combining and transforming vault values with Go templates (Sprig-flavored).

Template engine versions

Two engines coexist: v1 (legacy, default-when-unspecified for backwards compatibility) and v2 (active, recommended). Always set engineVersion: v2 in new manifests — v1 is deprecated and has fewer functions.

Example 1 — Dockerconfig JSON for image pull secrets

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: ghcr-pull
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: vault-store
    kind: SecretStore
  target:
    name: ghcr-pull
    template:
      type: kubernetes.io/dockerconfigjson
      engineVersion: v2
      data:
        .dockerconfigjson: |
          {
            "auths": {
              "ghcr.io": {
                "username": "{{ .username }}",
                "password": "{{ .token }}",
                "auth": "{{ printf "%s:%s" .username .token | b64enc }}"
              }
            }
          }
  data:
  - secretKey: username
    remoteRef: { key: ci/ghcr, property: username }
  - secretKey: token
    remoteRef: { key: ci/ghcr, property: token }
```

Example 2 — Postgres connection string

```
target:
  name: db-config
  template:
    engineVersion: v2
    data:
      DATABASE_URL: "postgres://{{ .user }}:{{ .password }}@{{ .host
}}:5432/{{ .dbname }}?sslmode=require"
data:
- secretKey: user
  remoteRef: { key: prod/db, property: user }
- secretKey: password
  remoteRef: { key: prod/db, property: password }
- secretKey: host
  remoteRef: { key: prod/db, property: host }
- secretKey: dbname
  remoteRef: { key: prod/db, property: dbname }
```

Example 3 — Build a TLS secret from PEM cert + key

```
target:
  name: ingress-tls
  template:
    type: kubernetes.io/tls
    engineVersion: v2
    data:
      tls.crt: "{{ .cert }}"
      tls.key: "{{ .key }}"
data:
- secretKey: cert
  remoteRef: { key: prod/tls, property: cert }
- secretKey: key
  remoteRef: { key: prod/tls, property: key }
```

Available template functions

ESO v2 ships hundreds of functions from Sprig plus a custom batch:

- **Encoding:** b64enc, b64dec, urlEncode, urlDecode
- **Crypto:** pkcs12cert, pkcs12key, fullPemToPkcs12, pemToPkcs12, jwkPublicKeyPem, jwkPrivateKeyPem
- **String:** upper, lower, trim, replace, regexReplaceAll, split, contains, hasPrefix, hasSuffix
- **JSON:** toJson, fromJson, get (gjson)
- **Conditional:** default, ternary, eq, ne, gt, lt
- **Math/list:** add, sub, len, first, last, slice

> 💡 **TIP**
> Use `default` to make secrets robust against missing fields: `{{ .api_key | default "missing-key" }}`. The

pod boots with a placeholder rather than crash-looping when the vault has not been populated yet.

16. Generators — Synthesizing Secrets

A generator creates a secret rather than reading one. Use a generator when the value should be machine-produced — random passwords, short-lived cloud tokens, dynamically issued certs. ESO ships 16+ generators; most fall into three families: random data, cloud-API short-lived tokens, and external service tokens.

Generator categories

Family	Generators	Use Case
Random	Password, UUID, SSHKey	Bootstrap accounts with unique creds
Cloud short-lived	AWS STS, AWS ECR, GCP GCR, Azure ACR	Image pull secrets that expire and refresh
External tokens	GitHub (App token), Grafana, Quay, Cloudsmith	API tokens for third-party services
Vault dynamic	Vault Dynamic Secret	Vault dynamic backends (DB, AWS, ...) — fresh creds per pull
Misc	Webhook, Fake	Custom HTTP source / testing

Example: Password Generator + PushSecret to AWS

A common pattern: generate a random database password inside the cluster, store it both as a Kubernetes Secret (for the workload) and in AWS Secrets Manager (for DR / cross-region failover). The password lives in two places, generated once, never typed.

```
apiVersion: generators.external-secrets.io/v1alpha1
kind: Password
metadata:
  name: db-password-gen
  namespace: my-app
spec:
  length: 32
  digits: 5
  symbols: 5
  symbolCharacters: "!@#$%^&*()_+"
  noUpper: false
  allowRepeat: false

---
# Pull the generated password into a K8s Secret
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: db-password
  namespace: my-app
spec:
```

```
  refreshInterval: "0"                  # Generate once, never refresh
  target:
    name: db-password
    creationPolicy: Owner
  dataFrom:
  - sourceRef:
      generatorRef:
        apiVersion: generators.external-secrets.io/v1alpha1
        kind: Password
        name: db-password-gen

---
# Push the same value into AWS Secrets Manager for backup
apiVersion: external-secrets.io/v1alpha1
kind: PushSecret
metadata:
  name: backup-db-password
  namespace: my-app
spec:
  refreshInterval: 1h
  secretStoreRefs:
  - name: aws-store
    kind: SecretStore
  selector:
    secret:
      name: db-password
  data:
  - match:
      secretKey: password
      remoteRef:
        remoteKey: prod/my-app/db-password
```

⚠ WARNING

A generator with refreshInterval > 0 will regenerate the value on every refresh, breaking your app. For random data that should be stable, set refreshInterval: "0" (generate once) or use creationPolicy: Orphan (never re-touch).

Example: ECR generator for short-lived image pull secrets

AWS ECR auth tokens expire every 12 hours. Manual rotation is a nuisance; the ECR generator handles it automatically — re-issuing a fresh token before each expiry.

```
apiVersion: generators.external-secrets.io/v1alpha1
kind: ECRAuthorizationToken
metadata:
  name: ecr-token
  namespace: my-app
spec:
```

```
  region: eu-central-1
  auth:
    jwt:
      serviceAccountRef:
        name: external-secrets-sa        # IRSA-enabled SA with
ecr:GetAuthorizationToken

---
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: ecr-pull
  namespace: my-app
spec:
  refreshInterval: 6h                    # Refresh well before 12h expiry
  target:
    name: ecr-pull
    template:
      type: kubernetes.io/dockerconfigjson
      engineVersion: v2
      data:
        .dockerconfigjson: |
          { "auths": { "{{ .proxy_endpoint }}": { "auth": "{{ .authorization_token
}}" } } }
  dataFrom:
  - sourceRef:
      generatorRef:
        apiVersion: generators.external-secrets.io/v1alpha1
        kind: ECRAuthorizationToken
        name: ecr-token
```

17. Multi-Tenancy & RBAC

A single ESO install routinely serves many teams. The challenge: keep team A from reading team B's secrets. ESO offers three layers of isolation — namespace scoping, controller classes, and policy engines.

Layer 1 — Namespace isolation (default)

ESO enforces a strict rule: an ExternalSecret in namespace foo can only reference a SecretStore in namespace foo or a ClusterSecretStore. Cross-namespace SecretStore references are blocked at admission time. Teams that stay within their own namespace are isolated by default.

Layer 2 — Restrict ClusterSecretStore via conditions

If you use ClusterSecretStores at all, restrict them. The conditions block lets you say "only these namespaces, or only namespaces with these labels can use me":

```
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: shared-vault
spec:
  conditions:
  - namespaces:
    - team-a
    - team-b
  - namespaceSelector:
      matchLabels:
        eso-tier: "shared"
  provider:
    vault: { ... }
```

Layer 3 — Controller classes (multi-instance ESO)

In rigorous multi-tenant environments, run multiple ESO controllers — one per tenant or environment. Each is given a controllerClass via the --controller-class flag and only reconciles resources whose .spec.controller field matches:

```
# Install two ESO instances side by side
helm install eso-prod external-secrets/external-secrets \
  -n eso-prod --create-namespace \
  --set controllerClass=prod

helm install eso-staging external-secrets/external-secrets \
  -n eso-staging --create-namespace \
  --set controllerClass=staging

# SecretStores opt in to one or the other
apiVersion: external-secrets.io/v1
kind: SecretStore
metadata:
  name: my-store
```

```
spec:
  controller: prod              # Only eso-prod will reconcile this
  provider: { ... }
```

Layer 4 — Policy engines

For organizational guardrails — "no team is allowed to use AWS in this cluster", "the only allowed provider is Vault", "you may not reference ClusterSecretStores from production namespaces" — use Kyverno or OPA Gatekeeper. These run as admission webhooks alongside the ESO webhook and reject manifests that violate policy.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: only-vault-provider
spec:
  validationFailureAction: Enforce
  rules:
  - name: deny-non-vault-providers
    match:
      any:
      - resources:
          kinds: [SecretStore, ClusterSecretStore]
    validate:
      message: "Only the HashiCorp Vault provider is permitted."
      pattern:
        spec:
          provider:
            vault: "?*"           # Wildcard: any value, but the key must exist
```

RBAC checklist

- Restrict ClusterExternalSecret and ClusterSecretStore creation to platform admins only.
- Application teams get edit on ExternalSecret in their own namespaces only.
- Audit the ESO controller ServiceAccount — it has read access to every Secret in every namespace by default.
- If using scopedRBAC: true and scopedNamespace: <ns>, ESO is locked to one namespace and ClusterSecretStore is disabled.

18. Security Best Practices

ESO sits in a sensitive position: it has read access to every Secret in every namespace, and outbound access to whichever vaults you point it at. A compromised ESO is a serious incident. The following practices, drawn from the official security guide, reduce blast radius.

Pod and runtime security

- Run with the restricted Pod Security Standard (the default since v0.8.2). Do not relax this.
- Use readOnlyRootFilesystem, runAsNonRoot, and drop all capabilities.
- Pin the image to a specific tag; verify the Cosign signature on every upgrade.
- Run at least two replicas of the controller and webhook for HA. Set a priorityClassName so they survive node pressure.

Verify image signatures

```
# Verify the ESO container image is genuine
COSIGN_EXPERIMENTAL=1 cosign verify \
  ghcr.io/external-secrets/external-secrets:v0.10.0 | jq

# Look at .[0].optional.Subject and .[0].optional.Issuer
# Should be:
#   Subject:
https://github.com/external-secrets/external-secrets/.github/workflows/
release.yml@refs/heads/main
#   Issuer:  https://token.actions.githubusercontent.com

# Verify SLSA provenance
COSIGN_EXPERIMENTAL=1 cosign verify-attestation --type slsaprovenance \
  ghcr.io/external-secrets/external-secrets:v0.10.0
```

Network policies — restrict egress

ESO needs egress only to (a) the Kubernetes API server, and (b) your secret providers. Block everything else with a NetworkPolicy. This prevents a compromised ESO from being used to exfiltrate data to attacker-controlled endpoints.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: external-secrets-egress
  namespace: external-secrets
spec:
  podSelector: { matchLabels: { app.kubernetes.io/name: external-secrets } }
  policyTypes: [Egress]
  egress:
  - to:                                    # Allow K8s API
    - namespaceSelector:
        matchLabels: { kubernetes.io/metadata.name: kube-system }
    ports: [{ protocol: TCP, port: 443 }]
```

```
  - to:                                    # Allow your Vault
    - ipBlock: { cidr: 10.0.5.0/24 }
    ports: [{ protocol: TCP, port: 8200 }]
  - to: []                                 # Allow DNS
    ports: [{ protocol: UDP, port: 53 }]
```

⚠ WARNING

If you do not lock down egress, ESO is a potential exfiltration vector. An attacker who can create an ExternalSecret with a malicious webhook provider URL can ship secrets out of your cluster. NetworkPolicies are the most important defense-in-depth control here.

Scope ServiceAccount token creation

By default, ESO has serviceaccounts/token: create cluster-wide — needed for Vault Kubernetes auth, Conjur, GCP WI. If compromised, ESO could mint tokens for any SA. Disable the blanket grant with rbac.serviceAccountTokenCreate: false in Helm values, then add per-SA Role/RoleBinding only where ESO genuinely needs it:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: eso-token-vault-sa
  namespace: my-app
rules:
- apiGroups: ['']
  resources: ['serviceaccounts/token']
  resourceNames: ['vault-sa']              # ONLY this SA
  verbs: ['create']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: eso-token-vault-sa
  namespace: my-app
subjects:
- kind: ServiceAccount
  name: external-secrets
  namespace: external-secrets
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: eso-token-vault-sa
```

Disable features you do not use

Helm flags to turn off reconcilers and CRDs you do not need. Smaller attack surface, less code running, less to audit.

```
# values.yaml — disable cluster-wide resources entirely
```

```
processClusterExternalSecret: false
processClusterStore: false
processPushSecret: false              # If you only pull, never push

crds:
  createClusterExternalSecret: false
  createClusterSecretStore: false
  createPushSecret: false
```

19. Observability — Metrics, Logs, Events

ESO is well-instrumented. Three signals matter day-to-day: Prometheus metrics for trend, Kubernetes events for incidents, and structured logs for debugging.

Prometheus metrics

Enable the ServiceMonitor in Helm and the operator starts publishing to your kube-prometheus-stack automatically. The interesting metrics:

Metric	Meaning	Alert When
externalsecret_sync_calls_total	Total sync calls (good for rate)	Drops to 0 unexpectedly
externalsecret_sync_calls_error	Failed sync attempts	Rate > 0 for > 5m
externalsecret_status_condition	Ready/Failed status per ES	Failed = 1 for > 10m
controller_runtime_reconcile_errors_total	Generic controller errors	Increasing
vault_secrets_total	Per-provider call counts	Sudden spike (likely a loop)
externalsecret_provider_api_calls_count	API calls broken down by provider	Approaching API rate limit

Kubernetes events

ESO emits events on every resource it manages. The first thing to check when something looks broken:

```
kubectl describe externalsecret my-app-secrets -n my-app

# Useful sections:
#   Status.Conditions — Ready=True is the happy path
#   Status.RefreshTime — when did ESO last successfully sync?
#   Events — chronological history of what the controller did
```

Logs

JSON-formatted by default. Key fields: msg, error, resourceVersion, namespace, ExternalSecret. Grep for level=error in the operator logs to find anything not exposed via metrics.

```
kubectl logs -n external-secrets deployment/external-secrets --tail=200 \
  | jq 'select(.level=="error")'
```

CloudEvents / external notifications

ESO does not natively emit CloudEvents (unlike KEDA). For audit pipelines, scrape the Kubernetes events stream (the kubernetes-event-exporter project is a good fit) or watch the ExternalSecret resource directly via a CronJob and a kubectl-to-webhook script. PRs are welcome upstream for native CloudEvent support.

20. Real-World Patterns & Use Cases

A grab-bag of patterns proven in production. Each pattern names the problem, sketches the solution, and points at the relevant CRDs.

Pattern 1 — Centralized DB password rotation

You rotate the production DB password every 30 days as part of a compliance requirement. Without ESO: open a ticket, coordinate with twelve teams, redeploy. With ESO: rotate in Vault, wait one refreshInterval, restart pods (or use a config reloader). Total elapsed time goes from 4 hours to 60 seconds.

Pattern 2 — Bootstrapping a brand-new cluster

You stand up a fresh GKE cluster for a new region. The cluster needs database creds, API tokens, TLS certs — twenty different secrets. Without ESO: copy-paste from a runbook, miss one, debug for an hour. With ESO: install the chart, apply your standard SecretStore + ExternalSecret manifests from Git. All twenty secrets materialize in five minutes.

Pattern 3 — Generated random passwords stored everywhere

You need a Redis password. Nobody should ever know it; rotation should be trivial. Pattern: Password generator → ExternalSecret (creates K8s Secret) → PushSecret (writes to Vault for backup). The password is generated once, lives in the cluster, and the only "external" record is in Vault. To rotate: delete the K8s Secret. ESO regenerates and pushes the new one.

Pattern 4 — Image pull secrets from ECR

ECR tokens expire every 12 hours. Manually rotating dockerconfigjson is brittle. Pattern: ECRAuthorizationToken generator + ExternalSecret with template, refreshInterval: 6h. Image pull secrets are always fresh, never expire mid-deployment.

Pattern 5 — Migrating off Sealed Secrets

Sealed Secrets keeps encrypted secrets in Git. Useful, but rotation is painful and cross-cluster sync is non-existent. Migration path: stand up Vault → seed it from Sealed Secrets values → replace Sealed Secrets with ExternalSecrets pointing at Vault → delete the SealedSecret CRDs once cutover is verified. Plan for a few weeks of running both side-by-side.

Pattern 6 — Cross-cluster certificate sync

You run cert-manager on cluster A and need the same TLS cert on cluster B (geo-failover). Pattern: cert-manager → K8s Secret on cluster A → PushSecret to Azure Key Vault → ExternalSecret on cluster B pulling from the same Key Vault. Cert renewals happen on A; cluster B gets the update at the next refresh.

Pattern 7 — Per-environment secret namespacing in Vault

Vault paths: kv/dev/my-app/*, kv/staging/my-app/*, kv/prod/my-app/*. Each environment cluster has its own SecretStore with a different vault role granting access only to its prefix. The ExternalSecret manifest is identical across environments — only the SecretStore differs. GitOps-friendly.

Pattern 8 — Fan-out to many namespaces

Forty microservice namespaces all need a shared OpenTelemetry collector endpoint + token. Pattern: ClusterExternalSecret with namespaceSelector matchLabels: { otel-enabled: "true" }. Tag a namespace, get the secret automatically. Untag, the secret disappears.

Pattern 9 — Generator-driven service account credentials

A multi-tenant SaaS needs a unique service account per customer in a downstream API. Pattern: Webhook generator pointed at an HTTPS endpoint that mints accounts on demand → ExternalSecret per customer namespace. Account creation happens lazily on ExternalSecret creation, not at customer-onboarding time.

Pattern 10 — GitOps bootstrap loop (Flux + ESO)

Catch-22: your Flux installation needs a token to read a private Git repo, but the token lives in Vault, and you cannot reach Vault without first installing ESO from that same repo. Pattern: bootstrap a tiny manually-applied "bootstrap" secret with a one-time Flux token, use Flux to install ESO, then ESO syncs all other secrets including the real Flux token, then rotate the bootstrap token out. Standard chicken-and-egg of GitOps; ESO makes the rest of the egg-laying automatic.

21. Tips, Tricks, and Common Pitfalls

Tip 1 — Use creationPolicy: Merge for app-managed Secrets

When the application or an Operator also writes to the Secret (e.g., adding generated fields), use creationPolicy: Merge so ESO updates only its own keys and leaves the others alone. With Owner, ESO will fight the other writer and one of them loses.

Tip 2 — Set refreshInterval thoughtfully

1h is the default and right for most cases. Shorten to 5m only for actively-rotating secrets (DB passwords on a fast schedule, API tokens). Lengthen to 24h or 0 for genuinely static secrets — saves vault API costs.

Tip 3 — Validate vault payloads with kubectl events

When an ExternalSecret stays Pending or Failed, kubectl describe externalsecret <name> shows the exact error from the vault. "permission denied", "secret not found", "key does not exist": all visible there, no need to dig in operator logs.

Tip 4 — Use dataFrom.find sparingly

dataFrom.find with a regex like .* will pull every secret matching the path. Convenient for bootstrap but expensive at scale, and harder to audit ("what does this ES actually contain?"). Prefer explicit data declarations once you know the shape.

Tip 5 — Reload pods on secret change

ESO updates the K8s Secret, but most apps read env vars at startup and never re-read. Three solutions: (a) stakater/Reloader watches Secrets and triggers Deployment rollouts; (b) volume-mounted Secrets are updated in place by the kubelet, so apps that re-read files get rotation for free; (c) sidecars like vault-agent-injector handle SIGHUP. Pick one and document it for your team.

Tip 6 — Use templates for "everything-in-one-secret" patterns

Some apps (Confluence, certain old Java services) want a single config file rather than environment variables. Use a template to assemble all fields into one config.yaml or .env entry inside the K8s Secret, then mount it as a file.

Pitfall 1 — Forgetting audiences on Kubernetes auth (Vault 1.21+)

Vault 1.21 made the audience field on Kubernetes auth roles mandatory. Existing roles without an audience will silently issue warnings on 1.20 and fail outright on 1.21. Add audiences: [vault] to the role config and serviceAccountRef in your SecretStore.

Pitfall 2 — ClusterSecretStore namespace ambiguity

A ClusterSecretStore references credentials. If the credentials live in a K8s Secret, that Secret has a namespace. You MUST set namespace: <ns> on every secret reference inside the ClusterSecretStore — there is no defaulting. Forget it and the controller logs "secret not found" with no hint about where it looked.

Pitfall 3 — Race condition on first install

If you apply a SecretStore and ExternalSecret in the same kubectl apply -f bundle, ESO may reconcile the ExternalSecret before the SecretStore is healthy and mark it Failed. It usually self-corrects within one

refreshInterval, but in CI/CD pipelines this can fail health checks. Solution: apply SecretStore first, wait for Ready, then apply ExternalSecret.

Pitfall 4 — Secrets > 1 MB

Kubernetes hard-limits Secret size to 1 MB. AWS Secrets Manager allows 64 KB per secret; Vault allows up to ~1 MB per KV entry. If you store large blobs (machine learning model weights, big TLS bundles), the Kubernetes limit will bite first. Split into multiple secrets or use a different mechanism (ConfigMap, mounted volume, init container).

Pitfall 5 — Webhook downtime breaks new applies

When the ESO webhook pod is unavailable, kubectl apply on new ExternalSecrets is REJECTED — failure mode is "fail closed". Run two webhook replicas in production. Set a PodDisruptionBudget. Make sure node drains do not evict both at once.

Pitfall 6 — IRSA / WI not propagating

You set the ServiceAccount annotation, the ExternalSecret still fails with "no credentials". Usually one of: (a) pod was started before the annotation was added — restart the pod; (b) the operator pod is not in the same namespace as the SA — for cross-namespace, IRSA must be on the ESO operator pod, not the workload SA; (c) cluster does not have the OIDC issuer enabled — for EKS, check oidc.eks.<region>.amazonaws.com/id/... exists in IAM identity providers.

Pitfall 7 — Stale credentials in Vault token cache

If you rotate the Vault role policy, the cached tokens still have the old (broader) permissions until they expire. Disable the cache (--enable-vault-token-cache=false) during sensitive rotations, or shorten the role token_ttl so the blast window is small.

22. Further Learning & References

ESO has a deep documentation tree and an active community. Here are the most useful next steps, organized roughly from getting-started to deep specialization.

Official documentation

- Introduction & guides: external-secrets.io
- API overview: introduction/overview
- ExternalSecret API spec: api/externalsecret
- SecretStore API spec: api/secretstore
- Full provider list: external-secrets.io/latest/provider
- Security best practices: guides/security-best-practices
- Threat model: guides/threat-model
- Multi-tenancy guide: guides/multi-tenancy
- Templating (v2): guides/templating
- Generators: guides/generator
- FAQ: introduction/faq

Provider deep-dives

- AWS Secrets Manager: provider/aws-secrets-manager
- AWS Parameter Store: provider/aws-parameter-store
- Azure Key Vault: provider/azure-key-vault
- HashiCorp Vault: provider/hashicorp-vault
- Google Cloud Secret Manager: provider/google-secrets-manager
- GitHub Actions Secrets: provider/github
- 1Password Connect: provider/1password-automation
- Kubernetes (cross-cluster): provider/kubernetes

Source code & community

- ESO GitHub: github.com/external-secrets/external-secrets
- Helm chart: deploy/charts/external-secrets
- Kubernetes Slack: kubernetes.slack.com/messages/external-secrets
- Community meetings calendar: contributing/calendar
- CNCF Sandbox page: cncf.io/projects/external-secrets-operator

Cloud-specific identity docs

- IAM Roles for Service Accounts (EKS): docs.aws.amazon.com IRSA
- AKS Workload Identity: learn.microsoft.com AKS WI
- GKE Workload Identity: cloud.google.com GKE WI
- Vault Kubernetes auth method: developer.hashicorp.com Vault K8s auth

Talks worth watching

ESO has a steady KubeCon presence. Useful talks (search by title on YouTube):

- "External Secrets Operator: A Practical Introduction" — Gustavo Carvalho, KubeCon EU. Best 30-minute on-ramp.

- "Managing Secrets at Scale" — Moritz Johner, recurring talk. Multi-tenant patterns, security model.
- "From Sealed Secrets to External Secrets" — migration war stories from production users.

Companion tools

- **stakater/Reloader:** Restart Deployments when their Secrets change. Pairs perfectly with ESO.
- **kyverno / opa-gatekeeper:** Policy engines to enforce SecretStore conventions.
- **cert-manager:** Issues TLS certs; combine with PushSecret to back them up.
- **esoctl:** Official CLI for debugging and bulk operations.

• esoctl docs: guides/using-esoctl-tool

Practice

- Spin up a kind cluster, install Vault dev mode (vault server -dev), install ESO via Helm, and walk through §12 end to end. About a half-day investment.
- Take a workload from your team that currently has secrets in Git. Migrate it to ESO. The first migration is the hardest; everything after is templating.
- Break things on purpose: delete a SecretStore, revoke an IAM permission, fill a vault with garbage. Observe how ESO reports the failure. Building that mental model pays off the first time a real incident hits.

Final thought

ESO is not the most exciting piece of infrastructure you will run. It is, however, one of the highest leverage. The day you stop having to think about how secrets get into pods is the day a class of incidents stops happening to your team. Get it installed, get one workload migrated, watch a rotation succeed automatically — and the rest follows.

About This Guide

This document is part of the **d-someone / tech-guide** project — a curated, open collection of practical technical guides authored and maintained on GitHub.

Feedback keeps these guides accurate and useful. If you spot something wrong, want a new topic covered, or have a question, please open an issue using one of the pre-filled templates below — it takes less than a minute.

> ✏️ **Found a mistake?**
> Submit a correction with the dedicated template: open a correction issue →
>
> 📚 **Want a guide on a different topic?**
> Request a new guide here: open a guide request →
>
> 💬 **Have a question?**
> Ask in the question tracker: open a question issue →

— d-someone · github.com/d-someone/tech-guide